



Surya Workshop Baselines

SOUTHWEST RESEARCH INSTITUTE®

Andrés Muñoz-Jaramillo

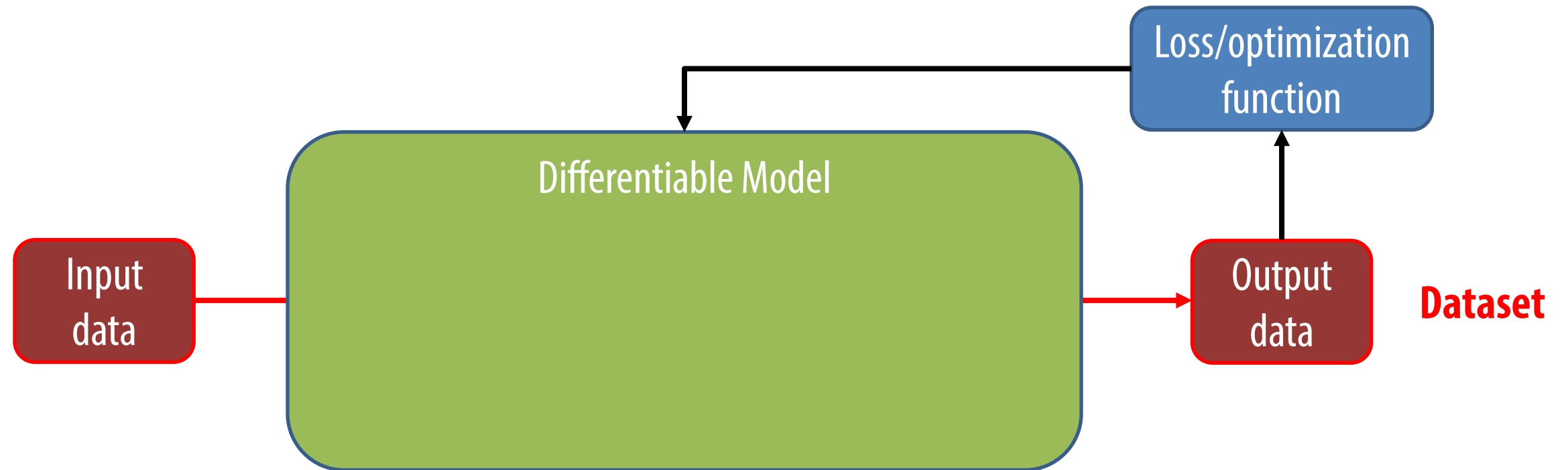
andres.munoz@swri.org

Commissioned and funded by NASA's Office of the Chief Science Data Officer (OCSDO)

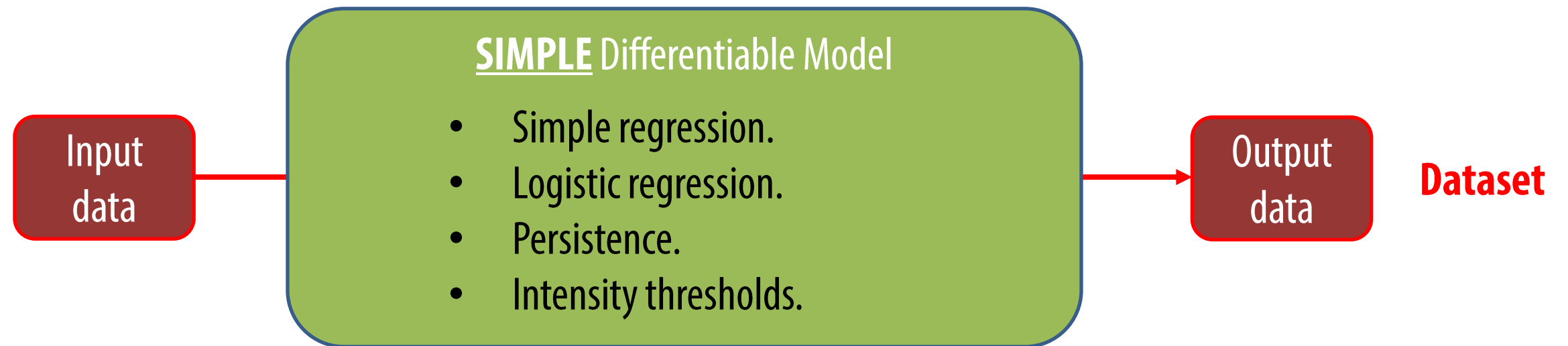


SOLAR SYSTEM SCIENCE & EXPLORATION

The elements of an AI training loop



The model



Basic Elements of a Model

- `__init__`

- Sets the differentiable mathematical operations that transform the input.
- Defines the learnable parameters

- forward

- Executes the mathematical transformation of the input into the output

Andres Munoz-Jaramillo, 16 hours ago | 1 author (Andres Munoz-Jaramillo)

```
class RegressionFlareModel(nn.Module):
```

```
def __init__(self, input_dim, channel_order, scalers):
```


Initializes the RegressionFlareModel.

Args:

`input_dim (int)`: The size of the input vector after channel and time

channel_order (list[str]): List of channel names, defining the order

This is used to ensure the inverse transfo

scalers (dict): A dictionary of scalars, one for each channel, used for

1111

```
super().__init__()
```

```
self.linear = nn.Linear(input_dim, 1)
```

```
self.channel_order = channel_order
```

```
self.scalers = scalers
```

```
def forward(self, x):
```

■■■■

Performs a forward pass through the model.

Args:

x (torch.Tensor): Input tensor of shape (b, c, t, w, h).

b - Batch size

c - Channels

t - Time steps

w - Width

h - Height



```
# Avoid mutating the caller's tensor
```

```
x = x.clone()
```

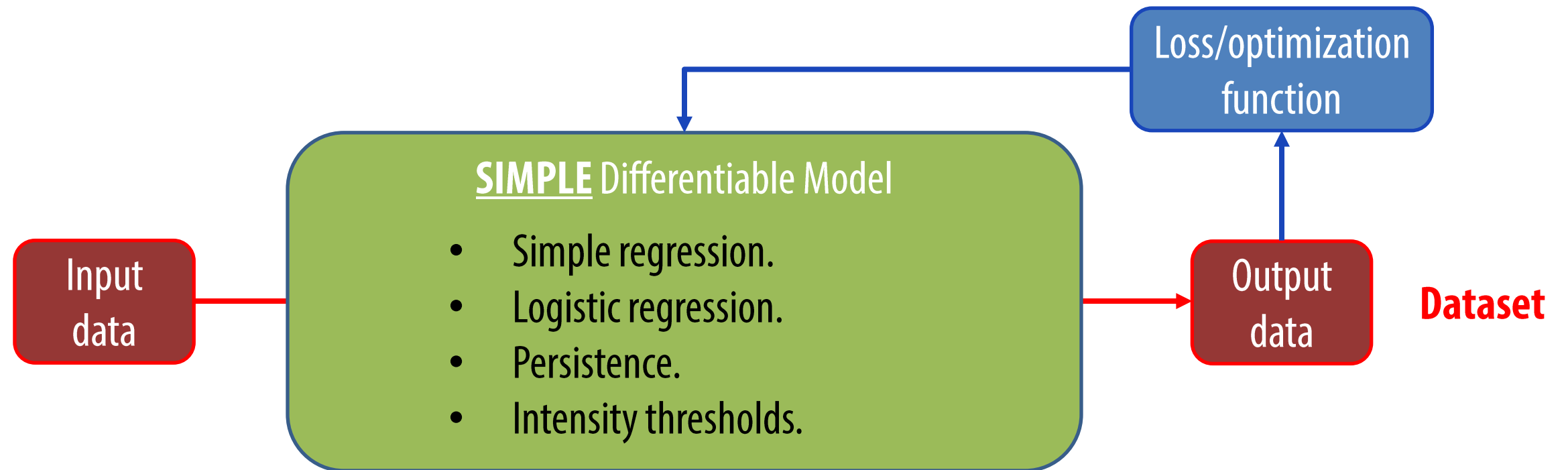
```
# Get dimensions
```

```
b, c, t, w, h = x.shape
```

```
with torch.no_grad():
```

```
for channel index, channel in enumerate(self.channel_order):
```

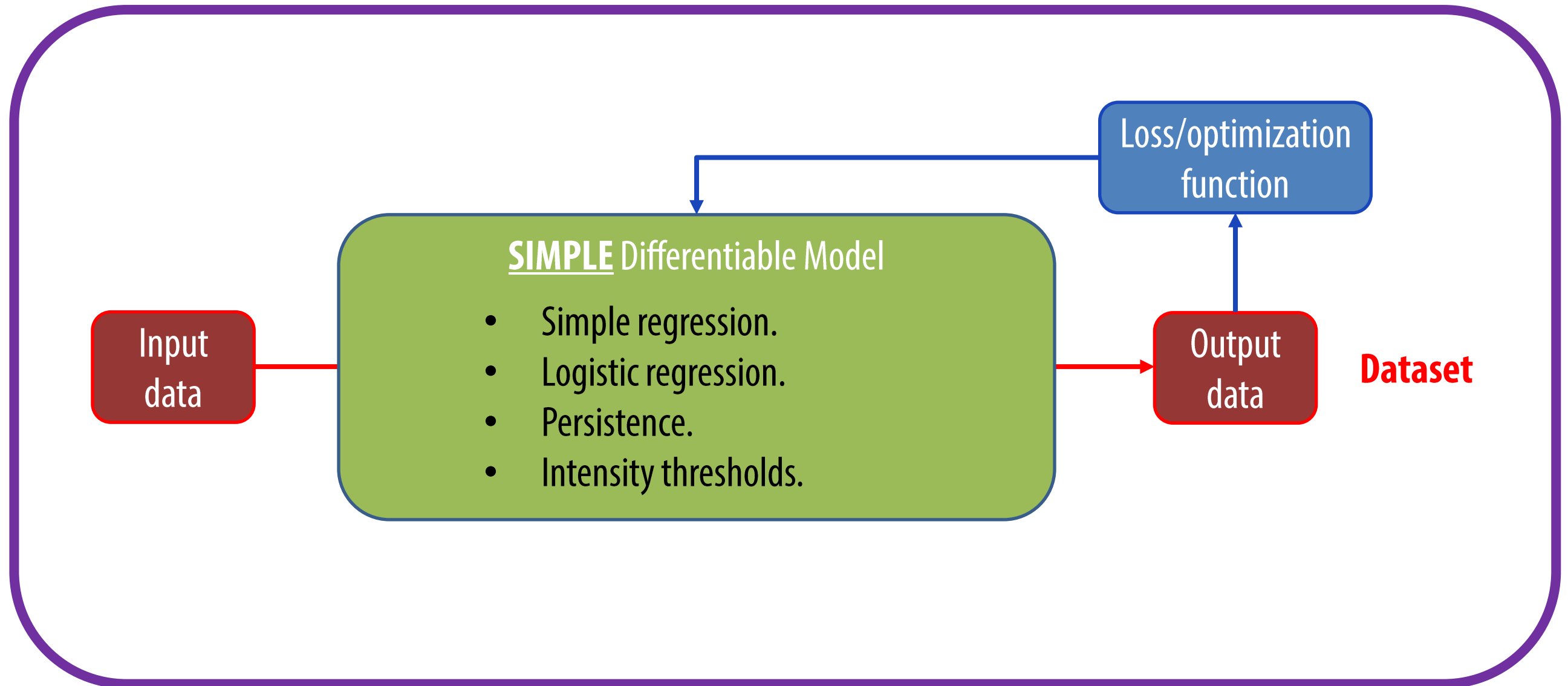
The metrics



The metrics

- Functions of the target data and the model output.
- Must be differentiable if you want to use them as losses.
- The loss can have multiple terms with different weights.
- Implemented in the template using dictionaries of losses.
- Are typically logged during training time.

The training loop: PyTorch lightning



Basic Elements of a Lightning module

- `__init__`
 - Initializes models and losses.
- Training and validation steps
 - Runs the model forward.
 - Calculates losses
 - Reports metrics.
- Configuration of optimizers
 - Sets gradient descent strategy.
 - Sets schedulers.

```
class MinimalLightningModule(L.LightningModule):  
    def __init__(self):  
        super().__init__()  
        # Define your model layers here (e.g., self.model = nn.  
        # For this minimal example, we just use a placeholder  
        self.save_hyperparameters() # Optional: saves __init__  
  
    def training_step(self, batch, batch_idx):  
        # Your training logic goes here  
        # For a minimal example, let's assume the batch is a s  
        x = batch  
        loss = torch.sum(x) * 0 # A dummy loss that is always  
        self.log("train_loss", loss) # Log the loss  
        return loss  
  
    def configure_optimizers(self):  
        # Configure your optimizer  
        # A dummy optimizer for the parameters (though none ar  
        return Adam(self.parameters(), lr=0.001)
```